UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Componentes de Lógica de Negocio · Unidad 4

Reporte Técnico

TechStore: núcleo de pedidos con patrones de lógica de negocio

Práctica Guiada — Actividad 4 · Desarrollo de un núcleo de negocio en Python

Docente: Miguel Ángel Gómez Marroquín

Grupo 004 · Semestre agosto–diciembre 2026

Equipo 9

Proyecto	TechStore — núcleo de pedidos
Integrantes	Jonathan Alexander Reyes García 2146387 Jorge Luis Molina García 2147723 Leonardo Dávila Góngora 2145945
Fecha de entrega	27/10/2026

Resumen ejecutivo

Este reporte documenta la construcción de TechStore, un núcleo de negocio para una tienda ficticia de tecnología. El sistema implementa entidades y objetos de valor, reglas de descuento componibles mediante el patrón Specification, una máquina de estados con eventos de dominio, un servicio de aplicación que carga los datos del paquete y una API REST construida con FastAPI. La solución se acompaña de una suite automatizada de 116 pruebas y de una bitácora reproducible de los 15 escenarios propuestos en el material de la práctica.

El punto de partida fueron cinco archivos CSV con 12 clientes, 12 productos, 16 pedidos y 21 renglones de detalle. Ningún archivo de datos fue editado y el cargador original (`cargar_csv.py`) se conservó intacto. Toda la lógica nueva se organizó en capas: dominio, Specifications, servicio de aplicación, adaptador HTTP y pruebas. El resultado es un proyecto ejecutable, verificable y documentado, con evidencias de cada paso.

Tabla 0. Resultados principales de la entrega

Indicador	Resultado
Pedidos cargados desde los CSV	16, todos inician en DRAFT
Escenarios de la bitácora	15 de 15 coinciden con el objetivo del paquete
Pruebas automatizadas	116 en verde
Endpoints de negocio	11
Endpoints de apoyo	2 (/health y /admin/reload-fixtures)
Archivos CSV modificados	0
Diagramas entregados	5, cada uno con fuente Mermaid, PlantUML, PNG y SVG

Índice

El índice se actualiza automáticamente al abrir el documento o al exportarlo a PDF. Si aparece desactualizado, use clic derecho sobre él y elija «Actualizar campo».

Resumen ejecutivo..... 2

Índice..... 3

1. Introducción..... 5

    1.1 Motivación..... 5

    1.2 Alcance..... 5

    1.3 Entregables..... 5

2. Objetivos..... 5

    2.1 Objetivo general..... 5

    2.2 Objetivos específicos..... 5

3. Contexto del problema..... 6

4. Análisis de los datos..... 6

    4.1 Catálogo de clientes..... 7

    4.2 Catálogo de productos..... 7

    4.3 Pedidos y totales..... 8

    4.4 Observaciones que condicionan el diseño..... 8

5. Arquitectura de la solución..... 8

    5.1 Estructura del repositorio..... 9

    5.2 Flujo de una petición..... 9

6. Modelo de dominio..... 10

    6.1 Invariantes del agregado..... 11

    6.2 Diagrama de clases..... 11

7. Patrón Specification..... 12

    7.1 Problema..... 12

    7.2 Diseño..... 12

    7.3 Reglas del negocio..... 12

    7.4 Composición y prueba..... 12

8. Máquina de estados y eventos..... 13

9. Servicio de aplicación..... 14

10. API REST 15
11. Pruebas y aseguramiento de calidad 16
11.1 Estrategia 17
11.2 Resultado de la ejecución 17
11.3 Casos de frontera 17
12. Escenarios E01–E15 18
12.1 Casos destacados 18
13. Evidencias y reproducibilidad 19
13.1 Carga de los datos CSV 19
13.2 Bitácora de escenarios E01–E15 19
13.3 Pruebas automatizadas 20
13.4 API REST en Swagger UI 20
13.5 Reproducibilidad 22
14. Decisiones de diseño y riesgos 23
14.1 Limitaciones 23
14.2 Trabajo futuro 23
15. Conclusiones 23
16. Referencias 24
Anexo A. Respuestas a las preguntas de la sección 15 25
Anexo B. Glosario 26
Anexo C. Comandos de reproducción 26
Link de GitHub 27

1. Introducción

1.1 Motivación

Las aplicaciones de comercio electrónico parecen sencillas desde la interfaz: elegir productos, confirmar un pedido y esperar la entrega. Detrás de esa secuencia hay reglas que no pueden quedar dispersas en los controladores. Un descuento mal calculado, una transición de estado permitida en el momento equivocado o un pedido modificado después de un rechazo producen pérdidas y decisiones difíciles de auditar. La práctica propone concentrar esas reglas en un núcleo de dominio explícito y verificable.

TechStore sirve como caso de estudio porque reúne tres problemas clásicos: valores monetarios que deben ser exactos, reglas comerciales que se combinan y un ciclo de vida con estados y eventos. La solución no depende de una base de datos ni de un marco web para expresar el negocio: el dominio puede probarse por sí solo y la API solo lo expone.

1.2 Alcance

El alcance de la entrega incluye el análisis de los datos, el modelo de dominio, las reglas de descuento, la máquina de estados, la API REST, la suite de pruebas, las evidencias de ejecución y la documentación técnica. Quedan fuera del alcance la persistencia en base de datos, la autenticación de usuarios, el descuento de inventario y el despliegue en producción. El repositorio es en memoria: los cambios en los pedidos se pierden al reiniciar el proceso, mientras que los CSV permanecen como fuente de verdad de solo lectura.

1.3 Entregables

- Código fuente organizado en `app/`, con dominio, Specifications, servicio y API.
- Suite de pruebas en `tests/`, con cobertura de dominio, API, escenarios y servicio.
- Bitácora de los 15 escenarios en Markdown, CSV y documento DOCX/PDF.
- Cinco diagramas UML en Mermaid, PlantUML, PNG y SVG.
- Evidencias reproducibles de carga, pruebas y respuestas HTTP.
- Este reporte técnico y el manifiesto de entrega con huellas SHA-256.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar un núcleo de lógica de negocio para TechStore que modele pedidos, clientes, productos y descuentos con reglas explícitas, estados controlados y una API REST verificable, aplicando buenas prácticas de separación de responsabilidades y pruebas automatizadas.

2.2 Objetivos específicos

1. Analizar la estructura y las relaciones de los cinco archivos CSV del paquete.
2. Representar importes monetarios con `Decimal` y objetos de valor inmutables.
3. Implementar las invariantes del pedido en el agregado `Order`.

4. Expresar las reglas de descuento como Specifications componibles con AND, OR y NOT.
5. Construir una máquina de estados con cinco transiciones y eventos de dominio.
6. Separar la carga de datos en un servicio de aplicación reutilizable.
7. Exponer el dominio mediante una API REST que traduzca excepciones a HTTP.
8. Verificar el comportamiento con pruebas de dominio, API, escenarios y frontera.
9. Documentar decisiones, errores, evidencias y resultados de forma reproducible.

3. Contexto del problema

Un pedido de TechStore recorre cinco estados: `DRAFT`, `CONFIRMED`, `SHIPPED`, `DELIVERED` y `RETURNED`; desde `CONFIRMED` también puede pasar a `CANCELLED`. Cada transición tiene precondiciones distintas. Confirmar exige que el pedido tenga artículos y que el cliente esté activo; enviar exige una guía no vacía; cancelar exige un motivo; entregar y devolver solo son válidos desde `SHIPPED`.

En paralelo, el negocio define dos descuentos. El 10 % aplica a clientes VIP con total mayor a 500 MXN, al menos cuatro unidades y sin promoción. El 5 % aplica a pedidos con total mayor a 5000 MXN y sin promoción. Si ambas reglas se cumplen, prevalece el 10 %. Los umbrales son estrictos: 500.00 exacto no califica para el 10 %, y el límite de unidades es 20, no 21.

El reto de diseño es que esas reglas se lean en un solo lugar y que ninguna operación inválida deje efectos parciales. Un rechazo no debe cambiar el estado, no debe agregar eventos y no debe modificar los artículos. Esa propiedad se verifica explícitamente en la suite.

4. Análisis de los datos

El paquete original contiene cinco archivos CSV. Antes de escribir código se revisaron las llaves, los tipos y las relaciones. La Tabla 4.1 resume el inventario; las tablas siguientes muestran los catálogos y los totales calculados para los 16 pedidos.

Tabla 4.1. Inventario de los datos originales

Archivo	Llave	Contenido	Registros
clientes.csv	cliente_id	Nombre, es_vip, activo	12
productos.csv	sku	Nombre, precio_mxn, existencia	12
pedidos.csv	pedido_id	cliente_id, tiene_promocion, estado_inicial	16
detalle_pedidos.csv	(pedido_id, sku)	Cantidad por renglón	21
escenarios.csv	caso_id	pedido_id, acción, objetivo, pista	15

4.1 Catálogo de clientes

Tabla 4.2. Los 12 clientes del paquete

ID	Nombre	VIP	Activo
C001	Ana Torres	Sí	Sí
C002	Luis Medina	No	Sí
C003	Carla Ruiz	Sí	No
C004	Diego Peña	No	No
C005	Sofia Lara	Sí	Sí
C006	Pedro Soto	No	Sí
C007	Mariana Gil	Sí	Sí
C008	Raul Vega	No	Sí
C009	Elena Mora	Sí	Sí
C010	Jorge Luna	No	Sí
C011	Valeria Rios	Sí	Sí
C012	Oscar Nieto	No	Sí

Dos clientes están inactivos (C003 y C004). Esa condición es la causa del rechazo de E03 y de una prueba específica de la API. Los clientes VIP se identifican con `es_vip = 1`; la regla del 10 % los exige, pero no basta por sí sola.

4.2 Catálogo de productos

Tabla 4.3. Los 12 productos del catálogo

SKU	Producto	Precio (MXN)	Existencia
SKU001	Mouse inalambrico	600.00	35
SKU002	Teclado mecanico	450.00	20
SKU003	Cable USB-C	120.00	70
SKU004	Audifonos	850.00	14
SKU005	Monitor 24 pulgadas	3,200.00	10
SKU006	SSD 1TB	1,750.00	18
SKU007	Webcam HD	950.00	22
SKU008	Laptop empresarial	18,000.00	7
SKU009	Hub USB	380.00	24
SKU010	Base para laptop	290.00	40
SKU011	Router Wi-Fi	2,100.00	11
SKU012	Memoria USB 64GB	160.00	50

La existencia es información de catálogo y no se descuenta al crear un pedido. El paquete no pide control de inventario; agregar esa regla habría cambiado el alcance y, probablemente, el resultado de varios escenarios.

4.3 Pedidos y totales

Tabla 4.4. Los 16 pedidos, con total calculado desde el detalle

Pedido	Cliente	VIP	Activo	Promo	Renglones	Unidades	Total
O001	Ana Torres	Sí	Sí	No	2	4	2,100.00
O002	Luis Medina	No	Sí	No	2	2	1,050.00
O003	Carla Ruiz	Sí	No	No	2	4	2,100.00
O004	Diego Peña	No	No	No	1	1	850.00
O005	Sofia Lara	Sí	Sí	Sí	2	4	2,100.00
O006	Pedro Soto	No	Sí	No	1	1	18,000.00
O007	Mariana Gil	Sí	Sí	No	0	0	0.00
O008	Raul Vega	No	Sí	No	1	20	2,400.00
O009	Elena Mora	Sí	Sí	No	1	19	2,280.00
O010	Jorge Luna	No	Sí	No	2	2	500.00
O011	Valeria Rios	Sí	Sí	No	2	3	1,500.00
O012	Oscar Nieto	No	Sí	No	1	1	850.00
O013	Ana Torres	Sí	Sí	No	1	1	600.00
O014	Luis Medina	No	Sí	No	1	1	600.00
O015	Sofia Lara	Sí	Sí	No	1	1	450.00
O016	Mariana Gil	Sí	Sí	No	1	1	2,100.00

Los totales se calcularon sumando cantidad por precio de catálogo. O001, O003 y O005 comparten el mismo detalle (2100.00 MXN) y se diferencian solo por el cliente o la promoción, lo que permite aislar el efecto de cada variable. O007 no tiene renglones y O008 llega exactamente a 20 unidades.

4.4 Observaciones que condicionan el diseño

- Las unidades no son renglones: O001 tiene 2 renglones y 4 unidades; O008 tiene 1 renglón y 20 unidades. El límite debe sumar cantidades.
- El total de O010 es exactamente 500.00, lo que obliga a distinguir `>` de `>=`.
- O011 tiene 3 unidades y un cliente VIP: sirve para probar que el umbral de unidades es independiente del monto.
- O006 es el único pedido con total mayor a 5000 y su cliente no es VIP; por eso la prioridad del 10 % sobre el 5 % se prueba con un pedido construido en memoria.
- O009 tiene 19 unidades; agregar dos más debe fallar sin modificar el pedido.
- Los escenarios E11 a E15 reutilizan pedidos que ya avanzaron de estado, así que cada caso debe ejecutarse sobre una carga nueva de los fixtures.

5. Arquitectura de la solución

La solución aplica una arquitectura por capas con dependencias dirigidas hacia el dominio. El núcleo no conoce FastAPI ni los archivos CSV; la API no repite reglas; el servicio es la única pieza que sabe cómo leer los datos del paquete.

Tabla 5.1. Capas y responsabilidades

Capa	Archivo	Responsabilidad	Depende de
Dominio	app/domain.py	Entidades, value objects, invariantes, estados y eventos	Solo biblioteca estándar
Reglas	app/specifications.py	Predicados de descuento y composición AND/OR/NOT	app.domain
Servicio	app/service.py	Lectura de CSV, construcción de pedidos y catálogos	app.domain
Adaptador	app/api.py	Endpoints, validación de entrada y traducción de errores	app.domain, app.specifications, app.service
Pruebas	tests/	Verificación del dominio, la API, el servicio y los escenarios	Todas las capas

5.1 Estructura del repositorio

```
techstore-u4/
├── app/
│   ├── __init__.py          # Documentación del paquete
│   ├── domain.py            # Entidades, value objects e invariantes
│   ├── specifications.py    # Reglas de descuento componibles
│   ├── service.py           # Carga de CSV y catálogos
│   └── api.py               # API REST FastAPI
├── tests/
│   ├── conftest.py
│   ├── test_domain.py
│   ├── test_specifications.py
│   ├── test_servicio.py
│   ├── test_escenarios.py
│   ├── test_maquina_estados.py
│   └── test_api.py
├── scripts/
│   ├── ejecutar_bitacora.py
│   ├── generar_evidencias.py
│   └── generar_documentos.py
├── datos/                   # Cinco CSV originales, sin editar
├── docs/                    # Análisis, bitácora, diagramas y evidencias
├── evidencias/              # Salidas reproducibles de la raíz
├── cargar_csv.py            # Cargador original, intacto
├── README.txt               # Reglas del paquete, intacto
└── requirements.txt
```

5.2 Flujo de una petición

Cuando llega una petición HTTP, FastAPI valida el cuerpo con Pydantic antes de invocar al controlador. El controlador busca el pedido en el diccionario en memoria, llama al método del agregado y, si el dominio lanza una excepción, la traduce a un código HTTP. El flujo es deliberadamente corto: la decisión de negocio no se reescribe en la capa web.

```
Cliente HTTP
    │
    ▼
FastAPI ── valida cuerpo (Pydantic) ──► 422 si el formato es inválido
    │
    ▼
Controlador ── busca el pedido ──► 404 si no existe
    │
    ▼
Order.confirm() / ship() / cancel() / deliver() / return_order()
    │         │
    │         └── valida precondiciones ──► DomainException ──► 409 / 422
    ▼
Cambia el estado y registra el DomainEvent
    │
    ▼
200 OK con el pedido serializado
```

6. Modelo de dominio

El modelo distingue entidades, objetos de valor y eventos. `Order` es el agregado raíz: todo cambio del pedido pasa por sus métodos. `Customer` tiene identidad y atributos que condicionan la confirmación. `Money` y `OrderItem` son inmutables y se comparan por valor.

Tabla 6.1. Elementos del modelo de dominio

Elemento	Tipo	Identidad	Responsabilidad
Money	Value object	No	Importe en MXN, suma y porcentaje con redondeo
OrderItem	Value object	No	SKU, cantidad, precio unitario y subtotal derivado
Customer	Entidad	cliente_id	Cliente VIP o activo, precondición de confirmación
Order	Aggregate root	pedido_id	Artículos, estado, eventos e invariantes
OrderStatus	Enumeración	—	DRAFT, CONFIRMED, SHIPPED, DELIVERED, RETURNED, CANCELLED
DomainEvent	Evento	—	Hecho ocurrido, con nombre y marca de tiempo
DomainException	Excepción base	—	Base de los rechazos de negocio

6.1 Invariantes del agregado

Tabla 6.2. Invariantes y excepciones

Invariante	Regla	Excepción
Importe válido	No se admiten importes negativos ni valores no finitos	InvalidOrderData
Artículo válido	SKU no vacío y cantidad mayor que cero	InvalidOrderData
Moneda	Los artículos del pedido deben estar en MXN	InvalidOrderData
Límite de unidades	La suma de cantidades no puede superar 20	OrderItemLimitExceeded
Edición	Solo se agregan artículos en DRAFT	InvalidOrderTransition
Confirmación	Requiere al menos un artículo y cliente activo	InvalidOrderData
Envío	Requiere guía no vacía y estado CONFIRMED	InvalidOrderData / InvalidOrderTransition
Cancelación	Requiere motivo no vacío y estado CONFIRMED	InvalidOrderData / InvalidOrderTransition
Entrega y devolución	Solo desde SHIPPED	InvalidOrderTransition

La validación ocurre antes de cualquier mutación. En `add_item()` se calcula primero el nuevo total de unidades y se lanza la excepción si excede el límite; solo después de validar se agrega el renglón. En las transiciones se comprueba el estado, luego la precondición, después se cambia `status` y al final se registra el evento. Esa secuencia garantiza que un rechazo no deje rastros.

6.2 Diagrama de clases

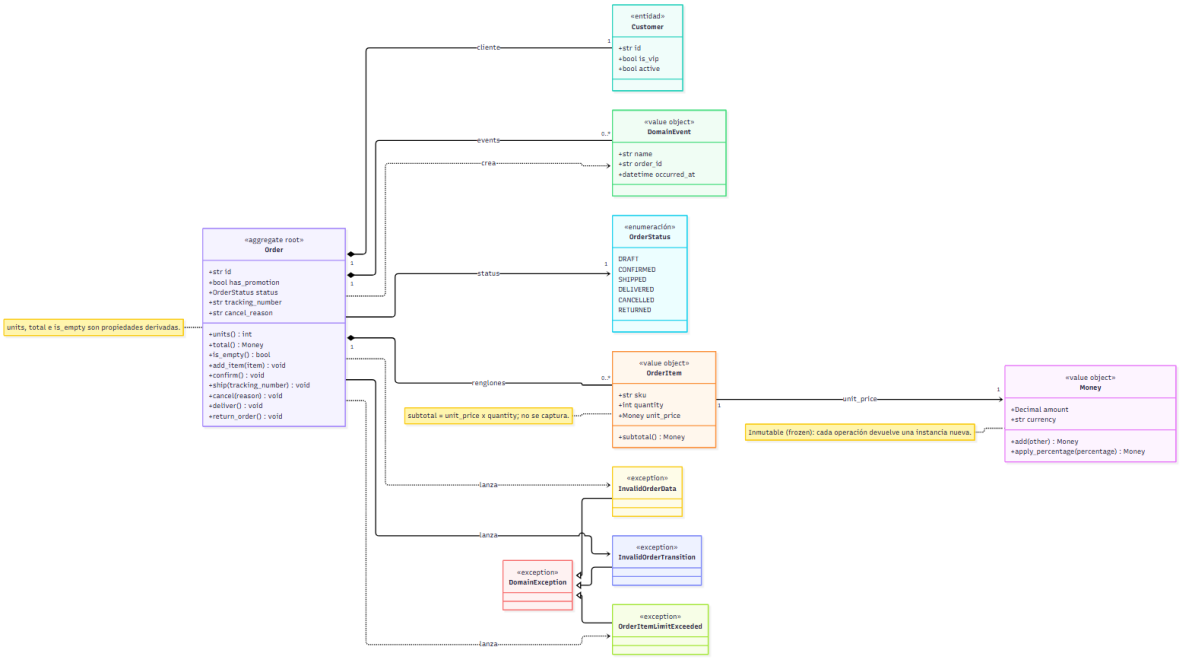


Figura 6.1. Entidades, value objects, enumeración, eventos y excepciones del dominio.

7. Patrón Specification

7.1 Problema

Las reglas de descuento combinan varias condiciones y cambian con el tiempo. Si se escriben como una cadena de `if`, cada modificación obliga a leer toda la función, las pruebas se acoplan al orden de las condiciones y la explicación del negocio queda escondida en el código. El patrón Specification convierte cada condición en un objeto con un método `is_satisfied_by(order)` y permite componerlas con operadores.

7.2 Diseño

Tabla 7.1. Componentes del patrón

Componente	Función
Specification	Interfaz base con `is_satisfied_by` y `description`
AndSpec / OrSpec / NotSpec	Composición con `&`, `|` y `~`
VIPCustomerSpec	Comprueba `customer.is_vip`
MinimumAmountSpec	Comprueba `order.total > umbral`
MinimumUnitsSpec	Comprueba `order.units >= umbral`
ActivePromotionSpec	Comprueba si el pedido tiene promoción
ActiveCustomerSpec	Comprueba que el cliente esté activo
HasItemsSpec	Comprueba que el pedido tenga al menos un renglón

7.3 Reglas del negocio

Tabla 7.2. Reglas de descuento

Regla	Condiciones	Descuento
TEN_PERCENT_RULE	VIP AND total > 500 AND unidades >= 4 AND NOT promoción	10 %
FIVE_PERCENT_RULE	total > 5000 AND NOT promoción	5 %
DISCOUNT_RULE	TEN_PERCENT_RULE OR FIVE_PERCENT_RULE	La mayor aplicable

La prioridad se resuelve en `resolve_discount`: si la regla del 10 % se cumple, se devuelve diez; en caso contrario se evalúa la del 5 %. El total bruto del pedido nunca se modifica. `evaluate_discount` devuelve un `DiscountResult` con el porcentaje, el total bruto, el total con descuento y el importe ahorrado.

7.4 Composición y prueba

Las Specifications se pueden combinar sin duplicar condiciones. Por ejemplo, la regla confirmable se expresa como la conjunción de «cliente activo» y «pedido con artículos». Además de las pruebas de resultado, la suite verifica que los objetos compuestos sean del tipo correcto y que las descripciones se deriven de los umbrales, no se escriban a mano.

```
TEN_PERCENT_RULE = (
    VIPCustomerSpec()
    & MinimumAmountSpec(500)
    & MinimumUnitsSpec(4)
    & ~ActivePromotionSpec()
)

FIVE_PERCENT_RULE = (
    MinimumAmountSpec(5000) & ~ActivePromotionSpec()
)

DISCOUNT_RULE = TEN_PERCENT_RULE | FIVE_PERCENT_RULE
```

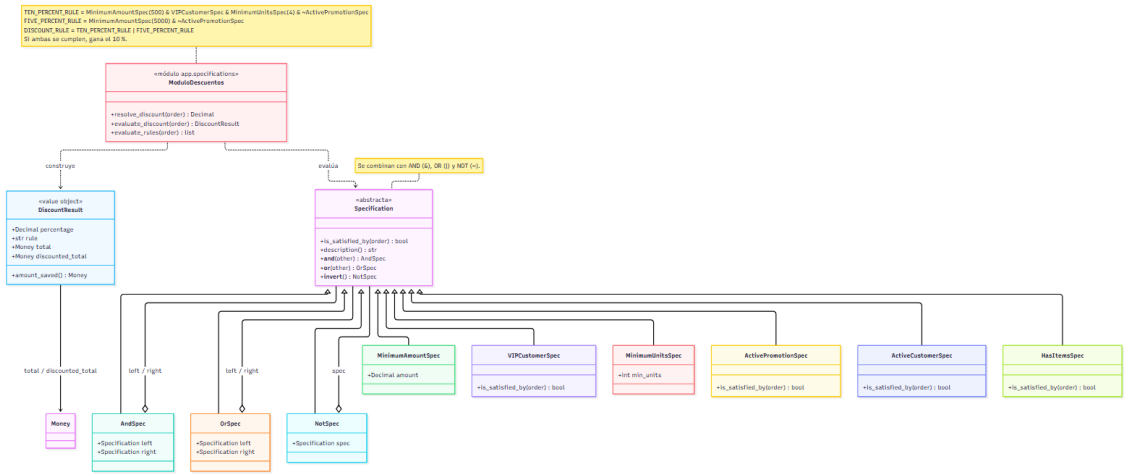


Figura 7.1. Composición de Specifications y cálculo del descuento.

8. Máquina de estados y eventos

El estado del pedido no es un campo libre: es una enumeración y solo cambia por los métodos del agregado. Cada transición válida registra un evento con nombre y marca de tiempo. Los eventos no se generan desde la API ni desde las pruebas; son consecuencia de la operación del dominio.

Tabla 8.1. Estados del pedido

Estado	Significado
DRAFT	Pedido en elaboración; admite artículos
CONFIRMED	Pedido aceptado; puede enviarse o cancelarse
SHIPPED	Pedido enviado con guía; puede entregarse o devolverse
DELIVERED	Pedido entregado al cliente (estado final)
RETURNED	Pedido devuelto (estado final)
CANCELLED	Pedido cancelado antes del envío (estado final)

Tabla 8.2. Transiciones permitidas

Desde	Hasta	Método	Precondición	Evento
DRAFT	CONFIRMED	confirm()	Con artículos y cliente activo	OrderConfirmed
CONFIRMED	SHIPPED	ship(guia)	Guía no vacía	OrderShipped
CONFIRMED	CANCELLED	cancel(motivo)	Motivo no vacío	OrderCancelled
SHIPPED	DELIVERED	deliver()	—	OrderDelivered
SHIPPED	RETURNED	return_order()	—	OrderReturned

La regla central es «validar, cambiar el estado y registrar el evento, en ese orden». Si la validación falla, el método lanza una excepción antes de tocar el estado. La suite comprueba el estado y la lista de eventos después de cada rechazo, no solo el tipo de excepción.

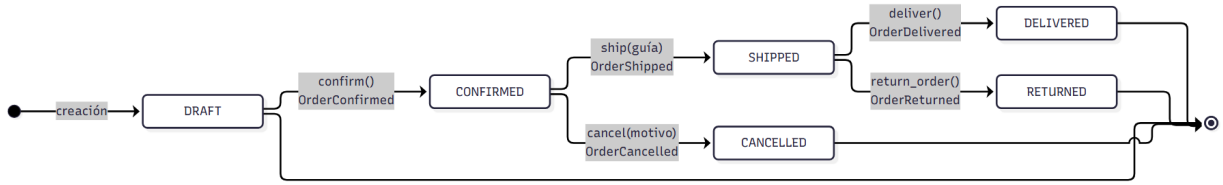


Figura 8.1. Máquina de estados de Order con sus cinco transiciones.

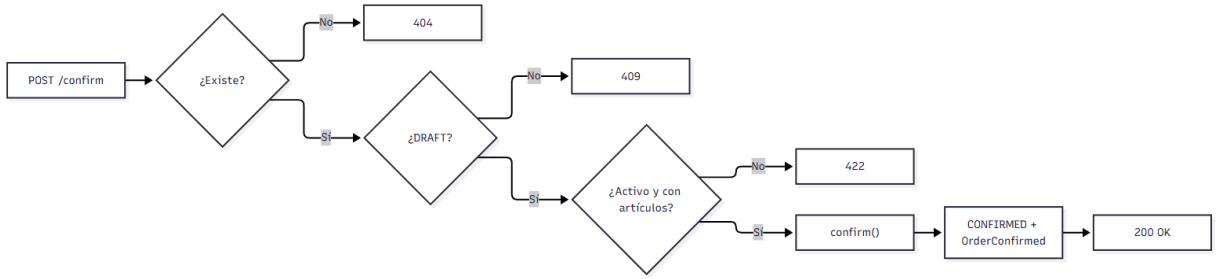


Figura 8.2. Decisión de POST /orders/{id}/confirm: 404, 409, 422 o 200.

9. Servicio de aplicación

`app/service.py` concentra la lectura de los CSV y la construcción de los objetos de dominio. La API y los scripts usan la misma función de carga, de modo que una diferencia entre la aplicación y las pruebas sería un error de una sola pieza, no de dos rutas paralelas.

Tabla 9.1. API del servicio

Función	Responsabilidad
read_rows(nombre)	Lee un CSV con codificación utf-8-sig y devuelve filas
load_orders(directorio)	Construye los 16 pedidos con clientes, productos y detalle
load_fixture(directorio)	Alias didáctico compatible con el cargador original
catalogo_productos()	Devuelve el precio unitario por SKU
catalogo_clientes()	Devuelve los datos de catálogo por cliente

El servicio acepta un directorio alternativo, lo que permite probar la carga con fixtures temporales sin tocar `datos/`. La API recarga los pedidos y los catálogos al arrancar y expone `POST /admin/reload-

fixtures` para repetir una demostración sin reiniciar el proceso. Esa recarga descarta los cambios en memoria, pero nunca escribe en los CSV.

10. API REST

La API se implementó con FastAPI. Los modelos de entrada validan el formato; las reglas de negocio siguen en el dominio. Los importes se serializan como cadenas para conservar la exactitud decimal en JSON.

Tabla 10.1. Endpoints de la aplicación

Método	Ruta	Descripción	Códigos
GET	/	Información general	200
GET	/orders	Resumen de los 16 pedidos	200
GET	/orders/{id}	Detalle con renglones, total y eventos	200, 404
GET	/orders/{id}/rules	Evaluación regla por regla	200, 404
POST	/orders/{id}/confirm	DRAFT → CONFIRMED	200, 404, 409, 422
POST	/orders/{id}/ship	CONFIRMED → SHIPPED	200, 404, 409, 422
POST	/orders/{id}/cancel	CONFIRMED → CANCELLED	200, 404, 409, 422
POST	/orders/{id}/deliver	SHIPPED → DELIVERED	200, 404, 409
POST	/orders/{id}/return	SHIPPED → RETURNED	200, 404, 409
POST	/orders/{id}/items	Agrega un renglón	200, 404, 409, 422
POST	/orders	Crea un pedido	201, 422
GET	/health	Estado del servicio	200
POST	/admin/reload-fixtures	Recarga los CSV	200

Los controladores comparten dos utilidades: una busca el pedido y lanza 404 si no existe; la otra traduce las excepciones del dominio. El mapeo es estable: `InvalidOrderTransition` produce 409, mientras que `InvalidOrderData` y `OrderItemLimitExceeded` producen 422. El 404 se decide en el controlador porque es una condición de la interfaz, no del agregado.

```
POST /orders/O013/confirm        -> 200 {"status": "CONFIRMED", ...}
POST /orders/O013/confirm        -> 409 {"detail": "..."}
POST /orders/O999/confirm        -> 404 {"detail": "Pedido no encontrado"}
POST /orders/O007/confirm        -> 422 {"detail": "Requiere artículos..."}
POST /orders/O009/items          -> 422 {"detail": "... 20 unidades ..."}
```

La documentación interactiva queda disponible en `/docs` y el contrato en `/openapi.json`. Las evidencias incluyen capturas reales de Swagger para 200, 404 y 422. La suite usa `TestClient` con el ciclo de vida de la aplicación, de modo que cada prueba parte de los 16 pedidos en DRAFT.

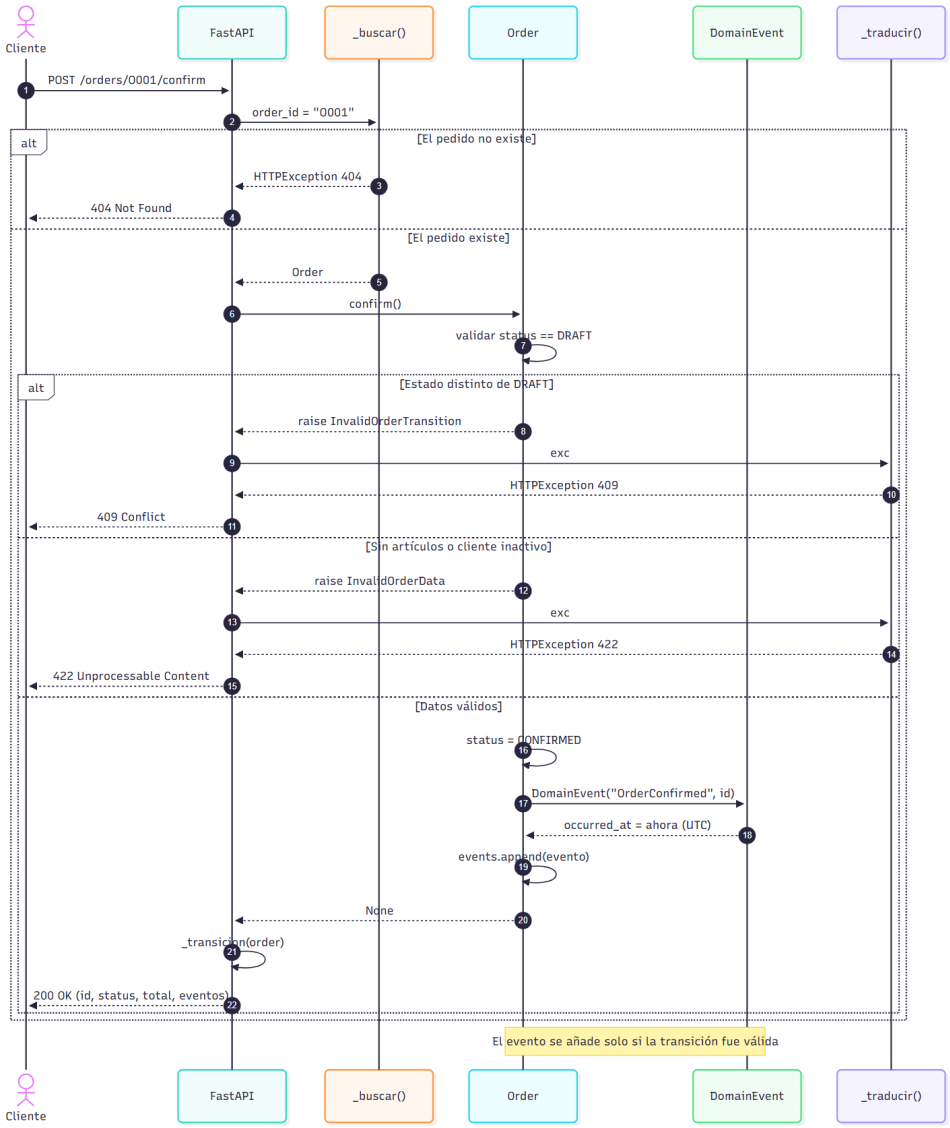


Figura 10.1. Secuencia de confirmación: controlador, agregado y traducción de errores.

11. Pruebas y aseguramiento de calidad

La suite se organiza por nivel de responsabilidad y suma 116 pruebas. Cada archivo prueba una capa distinta, aunque comparte los fixtures del paquete para que los datos sean los mismos que usa la aplicación.

Tabla 11.1. Distribución de las pruebas

Archivo	Pruebas	Cobertura
tests/test_domain.py	39	Money, OrderItem, invariantes y descuentos
tests/test_api.py	26	404, 409, 422 y ciclo de vida HTTP
tests/test_escenarios.py	15	Un caso por escenario E01–E15
tests/test_maquina_estados.py	21	Transiciones y eventos
tests/test_specifications.py	10	Specifications aisladas y compuestas
tests/test_servicio.py	5	Carga, catálogos, /health y recarga
Total	116	Suite completa en verde

11.1 Estrategia

- Las pruebas de dominio construyen objetos en memoria para controlar umbrales exactos, sin alterar los CSV.
- Las pruebas de API usan `TestClient` y verifican el código HTTP y el cuerpo.
- Las pruebas de escenarios cargan los fixtures antes de cada caso.
- Las pruebas de servicio comparan dos rutas de carga y comprueban los catálogos.
- Las pruebas de frontera cubren 500.00 exacto, 500.01, 3 y 4 unidades, 20 y 21.

11.2 Resultado de la ejecución

```
$ python -m pytest -q
.................................................................... [ 62%]
........................................                           [100%]
116 passed, 1 warning in 1.95s
```

La única advertencia proviene de `fastapi.testclient` por el uso de `httpx` con Starlette; no la genera el código del proyecto. Se documentó en lugar de silenciarla con un filtro, porque conocer su origen es parte de la entrega.

11.3 Casos de frontera

Tabla 11.2. Fronteras verificadas

Caso	Entrada	Resultado esperado
Monto estricto	Total 500.00 / 500.01	0 % / 10 % si cumple el resto
Unidades	3 / 4 unidades	Sin descuento / aplica si cumple el resto
Límite	20 / 21 unidades	Permitido / OrderItemLimitExceeded
Prioridad	VIP con total 6000.00 y 5 unidades	10 %, no 5 %
No mutación	Confirmar dos veces	409 y un solo evento
Sin guía	ship("")	422 y estado CONFIRMED

12. Escenarios E01–E15

Los escenarios se ejecutan con `scripts/ejecutar_bitacora.py`. Cada caso se carga desde cero, ejecuta la acción indicada y compara el resultado con el objetivo de `escenarios.csv`. La bitácora completa se entrega en Markdown, CSV y DOCX/PDF; aquí se resume el resultado.

Tabla 12.1. Resultado de los 15 escenarios

Caso	Pedido	Acción	Objetivo	Resultado observado	Estado final
E01	O001	evaluar_descuento	10%	10%	DRAFT
E02	O002	evaluar_descuento	0%	0%	DRAFT
E03	O003	confirmar	rechazo	RECHAZADO (InvalidOrderData)	DRAFT
E04	O005	evaluar_descuento	0%	0%	DRAFT
E05	O006	evaluar_descuento	5%	5%	DRAFT
E06	O007	confirmar	rechazo	RECHAZADO (InvalidOrderData)	DRAFT
E07	O008	confirmar	exito	CONFIRMED	CONFIRMED
E08	O009	agregar SKU003 x2	rechazo	RECHAZADO (OrderItemLimitExceeded)	DRAFT
E09	O010	evaluar_descuento	0%	0%	DRAFT
E10	O011	evaluar_descuento	0%	0%	DRAFT
E11	O012	confirmar y enviar sin guia	rechazo envio	RECHAZADO (InvalidOrderData)	CONFIRMED
E12	O013	confirmar dos veces	rechazo segunda	RECHAZADO (InvalidOrderTransition)	CONFIRMED
E13	O014	confirmar, enviar y cancelar	rechazo cancelacion	RECHAZADO (InvalidOrderTransition)	SHIPPED
E14	O015	confirmar, enviar y devolver	exito	RETURNED	RETURNED
E15	O016	confirmar, enviar y entregar	exito	DELIVERED	DELIVERED

Los 15 casos coinciden con el objetivo del paquete. Los rechazos conservan el estado previo y no agregan eventos; las transiciones válidas dejan exactamente el evento correspondiente. La coincidencia se calculó ejecutando el dominio, no escribiendo el resultado en la tabla.

12.1 Casos destacados

- E01 aplica el 10 % a O001: VIP, 2100.00 > 500, 4 unidades y sin promoción.
- E05 aplica el 5 % a O006: 18000.00 > 5000 y sin promoción.
- E08 rechaza agregar 2 unidades a O009 porque 19 + 2 = 21 supera el límite.
- E11 confirma O012 pero rechaza el envío sin guía; el evento OrderConfirmed permanece.
- E13 cancela desde SHIPPED y falla: la cancelación solo se permite desde CONFIRMED.
- E14 y E15 terminan en RETURNED y DELIVERED, respectivamente.

13. Evidencias y reproducibilidad

A continuación se presentan las capturas de pantalla de la ejecución real de cada paso: carga de los CSV, bitácora de escenarios, suite de pruebas y consumo de la API desde Swagger UI. Las salidas en texto se conservan además en `evidencias/` y `docs/evidencias/` para poder repetirlas.

13.1 Carga de los datos CSV

La carga de los cinco CSV construye los 16 pedidos en estado DRAFT con su número de artículos y su total.

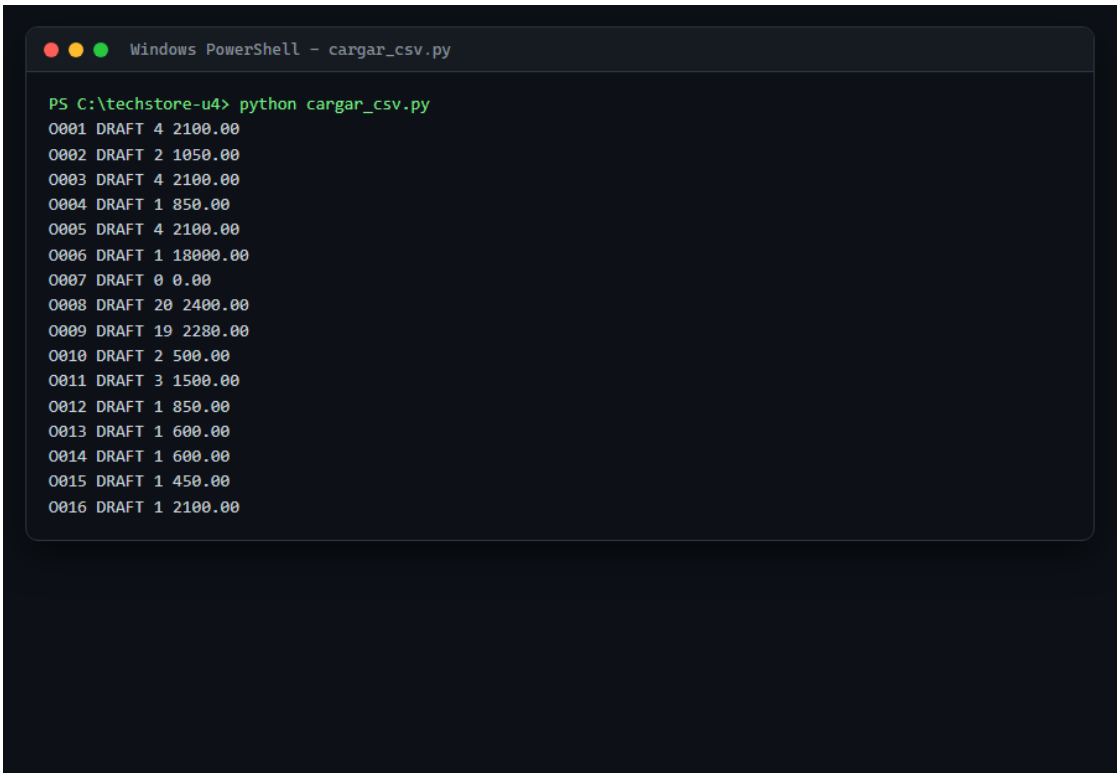


Figura 13.1. Ejecución de `python cargar_csv.py`: 16 pedidos cargados.

13.2 Bitácora de escenarios E01–E15

El script de bitácora ejecuta los 15 escenarios contra el dominio y compara el resultado observado con el esperado.

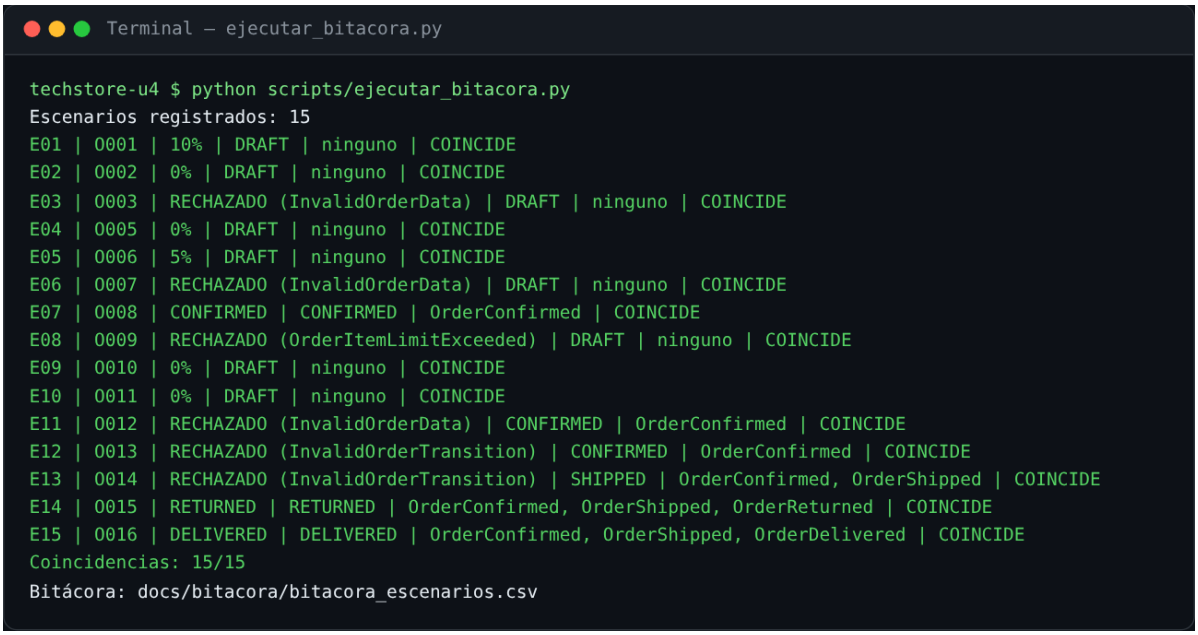


Figura 13.2. Ejecución de `python scripts/ejecutar_bitacora.py`: 15/15 coinciden.

13.3 Pruebas automatizadas

La suite completa termina en verde con 116 pruebas aprobadas.

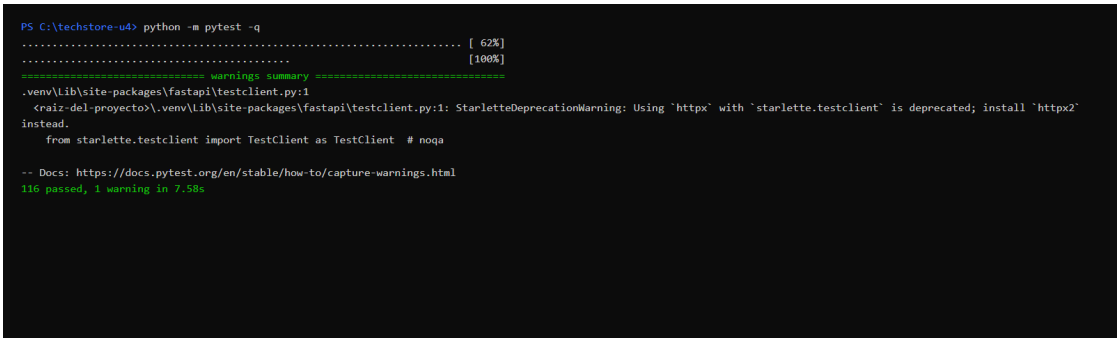


Figura 13.3. Ejecución de `python -m pytest -q`: 116 passed.

13.4 API REST en Swagger UI

Con `uvicorn app.api:app --reload` se levantó la API y se probaron los endpoints desde `/docs`, tanto en casos de éxito como de rechazo.

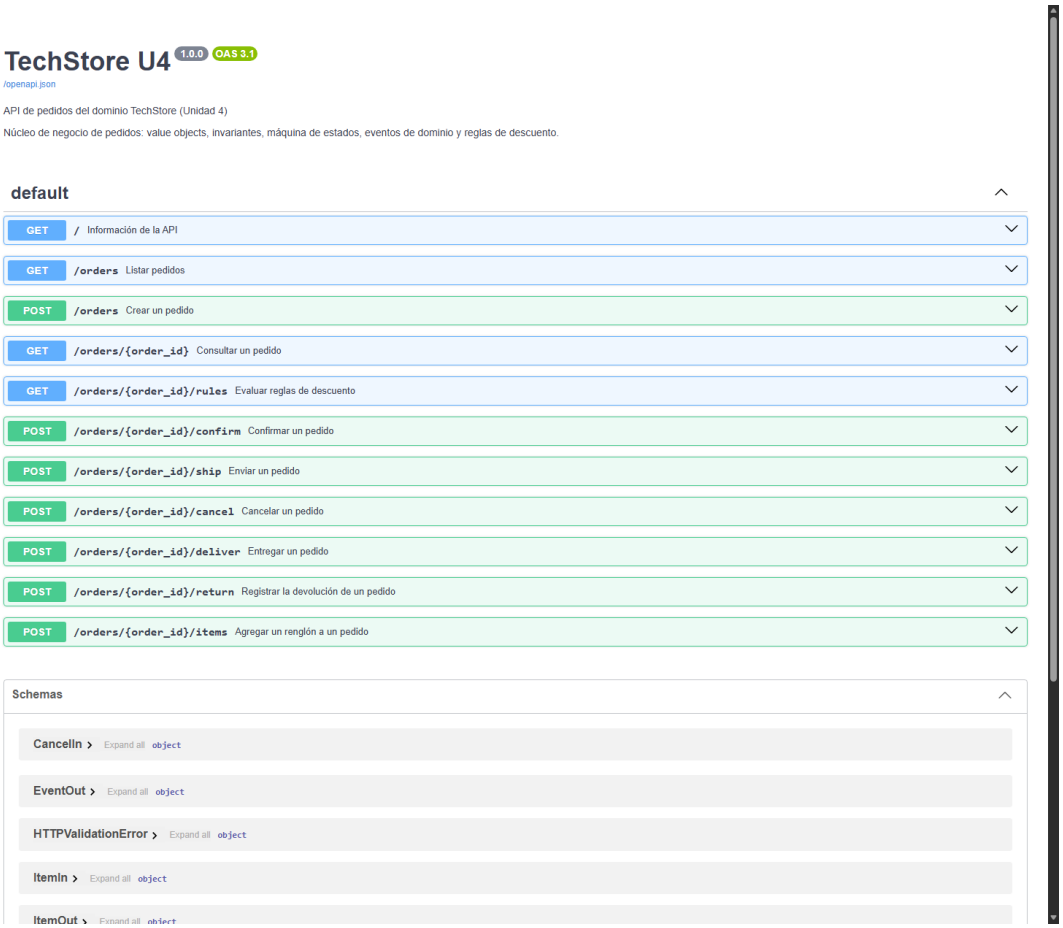


Figura 13.4. Documentación interactiva de la API en `/docs`.

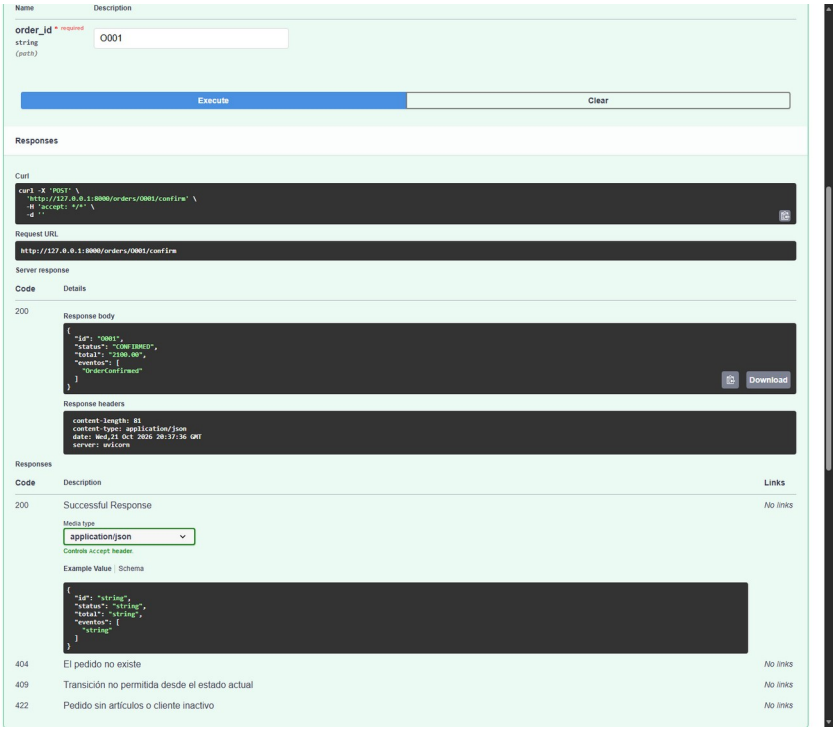


Figura 13.5. `POST /orders/O001/confirm`: respuesta 200 con el evento OrderConfirmed.

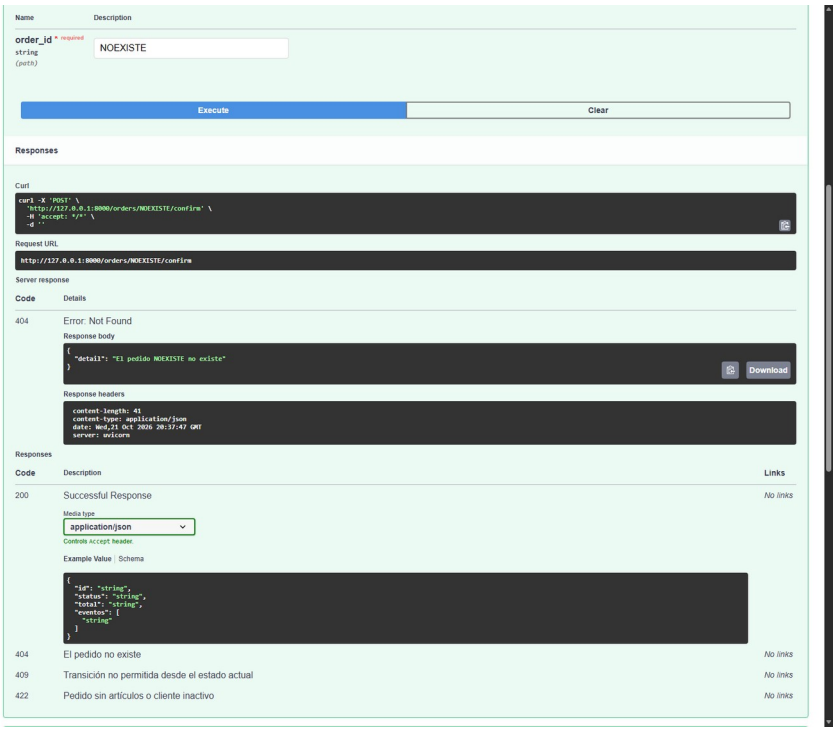


Figura 13.6. Rechazo 404: el pedido solicitado no existe.

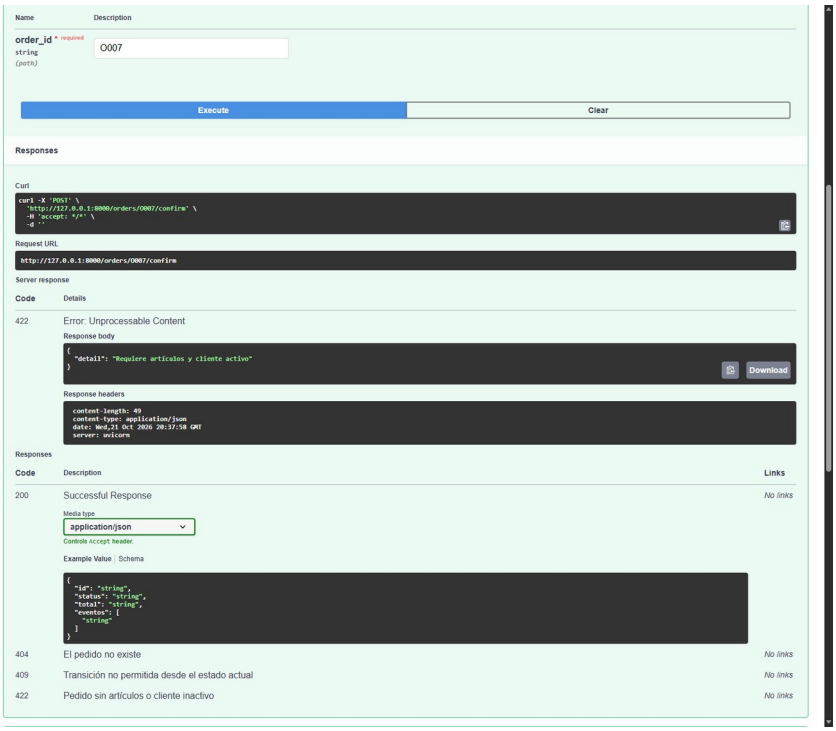


Figura 13.7. Rechazo 422: pedido sin artículos o cliente inactivo.

13.5 Reproducibilidad

Los mismos resultados se obtienen desde la raíz del proyecto con:

```
python cargar_csv.py
python scripts/ejecutar_bitacora.py
python -m pytest -q
uvicorn app.api:app --reload
```

14. Decisiones de diseño y riesgos

Tabla 14.1. Decisiones de diseño

Decisión	Alternativa descartada	Motivo
`Decimal` para importes	`float`	Evita errores de representación binaria
Descuento separado del total	Modificar `order.total`	El total bruto es un dato del pedido
Specifications componibles	Cadena de `if`	Reglas legibles, probables y reutilizables
Eventos registrados al final	Evento antes de validar	Un rechazo no debe dejar historia falsa
Servicio de aplicación	Carga dentro de la API	Una sola ruta de carga para API y scripts
Traducción de errores en el controlador	HTTPException en el dominio	El dominio no depende de FastAPI

14.1 Limitaciones

- El repositorio es en memoria; no hay persistencia entre reinicios.
- La autenticación y la autorización están fuera del alcance.
- No se descuenta inventario ni se integra un sistema de pagos.
- La recarga de fixtures descarta cambios locales, algo aceptable en el contexto académico.

14.2 Trabajo futuro

- Agregar persistencia con SQLAlchemy o un repositorio implementado sobre archivos.
- Incorporar autenticación y roles para distinguir clientes y administradores.
- Añadir métricas y trazas para observar el ciclo de vida de los pedidos.
- Ampliar la API con paginación y filtros por estado o cliente.

15. Conclusiones

La práctica permitió comprobar que un núcleo de negocio bien delimitado se puede probar y explicar sin depender de la interfaz. El dominio concentra las invariantes, las Specifications hacen explícitas las reglas comerciales y la máquina de estados garantiza que cada transición deje un rastro coherente. La API queda como un adaptador delgado, fácil de revisar.

Los 116 casos automatizados y los 15 escenarios reproducibles dan una base objetiva para afirmar que la entrega cumple el comportamiento esperado. El proceso también dejó aprendizajes sobre el orden de las anotaciones en Pydantic, la diferencia entre `>` y `>=`, el orden de las validaciones HTTP y la importancia de no modificar los datos para forzar un resultado.

El proyecto queda como un repositorio local listo para publicarse en GitHub, con manifiesto de entrega, documentación y comandos de reproducción. La siguiente ampliación natural es la persistencia, seguida de autenticación y observabilidad, sin mover las reglas fuera del dominio.

16. Referencias

- Documentación oficial de Python 3.14: tipos numéricos y módulo `decimal`.
- Documentación oficial de FastAPI: modelos Pydantic, dependencias y pruebas.
- Documentación oficial de pytest: fixtures, parametrización y captura de salida.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*.
- Fowler, M. (2013). *Catalog of Patterns of Enterprise Application Architecture*.
- Material de la práctica: `CLN_U4_Practica_Guiada_Alumnos_TechStore.pdf`.
- Repositorio del proyecto: `README.md`, `docs/` y `datos/`.

Anexo A. Respuestas a las preguntas de la sección 15

1. ¿Por qué se usa Decimal en lugar de float para precios?

Porque `float` representa los números en base dos y muchos importes decimales no tienen una representación binaria exacta. El clásico `0.1 + 0.2` produce `0.30000000000000004`. En un pedido, ese error se acumula y termina en un total que no coincide con la factura. `Decimal` trabaja en base diez, que es la forma en que se escriben los precios, y conserva la exactitud de las sumas y multiplicaciones. En el proyecto, `Money` normaliza la entrada con `Decimal(str(amount))` y redondea a centavos con `ROUND_HALF_UP`.

2. ¿Cómo se aplican las reglas de descuento del 10 % y del 5 %?

La regla del 10 % exige cuatro condiciones simultáneas: cliente VIP, total estrictamente mayor a 500 MXN, al menos cuatro unidades y ausencia de promoción. La del 5 % exige total estrictamente mayor a 5000 MXN y ausencia de promoción. Ambas se construyen como Specifications y se combinan con AND y NOT; la regla general es un OR de las dos. Si las dos se cumplen, `resolve_discount` devuelve primero el 10 %. El total bruto del pedido no se modifica: el resultado se entrega en un objeto aparte.

3. ¿Por qué el límite de 20 unidades se valida en el agregado y no en la API?

Porque es una invariante del pedido. Si se validara solo en la API, cualquier otro cliente del dominio (un script, una prueba o una futura interfaz) podría dejar un pedido con 21 unidades. El agregado es el único punto de entrada para modificar los artículos y por eso concentra la regla. La API se limita a traducir `OrderItemLimitExceeded` a 422. Además, la validación se hace antes de agregar el renglón, de modo que el rechazo no muta el pedido.

4. ¿Cómo se garantiza que una operación inválida no deje efectos parciales?

Cada método del pedido sigue el orden validar → cambiar el estado → registrar el evento. Si la validación falla, se lanza una excepción antes de tocar `status` o `items`. En `add_item` se calcula el nuevo total de unidades y se compara con el límite antes de agregar. Las pruebas verifican el estado, el número de eventos y los artículos después de cada rechazo, no solo el tipo de excepción. Así se detecta cualquier mutación accidental.

5. ¿Qué aporta separar el servicio de aplicación de la API?

El servicio es la única pieza que sabe leer los CSV y construir los pedidos. La API y los scripts comparten esa ruta, lo que evita divergencias entre la aplicación y las pruebas. También permite apuntar a un directorio alternativo de fixtures, recargar los datos sin reiniciar el proceso y sustituir la fuente de datos en el futuro sin reescribir los controladores. La API queda enfocada en HTTP y el servicio en la carga.

Anexo B. Glosario

Tabla B.1. Términos usados en el documento

Término	Definición
Value object	Objeto inmutable sin identidad, comparado por valor
Entidad	Objeto con identidad y ciclo de vida propio
Aggregate root	Entidad que protege las invariantes de un conjunto
Invariante	Condición que debe cumplirse siempre en el modelo
Specification	Predicado componible que expresa una regla
Domain event	Hecho ya ocurrido que se registra para auditoría
Fixture	Conjunto de datos de entrada usado en pruebas o demostraciones
Adapter	Capa que traduce entre el dominio y un mecanismo externo

Anexo C. Comandos de reproducción

```
# 1. Crear y activar el entorno
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

# 2. Ejecutar el cargador y la bitácora
python cargar_csv.py
python scripts/ejecutar_bitacora.py

# 3. Ejecutar todas las pruebas
python -m pytest -q

# 4. Regenerar evidencias y documentos
python scripts/generar_evidencias.py
C:\Python314\python.exe scripts/generar_documentos.py

# 5. Levantar la API
uvicorn app.api:app --reload
```

Link de GitHub

El código fuente, las pruebas, los datos originales, los diagramas, las evidencias y estos documentos están publicados en el repositorio del equipo:

https://github.com/leyesdavila/Componentes-Logica-de-Negocios-EQ9/tree/main/techstore-u4